COMMUNITY ITEM LENDING SYSTEM

A Web-Based Platform for Sharing and Borrowing Household Resources

MINI PROJECT REPORT

[TITLE PAGE — LEAVE AS BLANK I WILL FILL]

BONAFIDE CERTIFICATE

[BONAFIDE CERTIFICATE — LEAVE AS BLANK I WILL FILL]

ACKNOWLEDGEMENT

I wish to express my profound gratitude and sincere appreciation to our esteemed project guide and faculty coordinators for their continuous encouragement, insightful technical guidance, and valuable constructive suggestions throughout the course of this mini project.

I extend my heartfelt thanks to the Head of the Department and all respected faculty members of the Department of Computer Applications for providing excellent laboratory infrastructure, online server resources, and academic support that enabled the timely and successful development of the Community Item Lending System.

I would also like to thank my peers, teammates, and fellow community members whose active participation, constructive feedback, and helpful suggestions during user testing contributed substantially to enhancing the platform's reliability, user experience, and practical functionality.

Finally, I express my deepest indebtedness to my parents and family members for their constant moral support, patience, and encouragement during every phase of this academic work.

Candidate Name / Author
Department of Computer Applications

CONTENTS

1. INTRODUCTION....1
1.1 Need for the System....1
1.2 Objectives of the System....2
1.3 Scope of the Application....3
1.4 Technology Context and Agile Methodology....3
1.5 Organization of the Report....4
2. ANALYSIS & REQUIREMENTS....5
2.1 Existing Situation and Deficiencies....5
2.2 Proposed System and Innovations....6
2.3 Functional Requirements....6
2.4 Non-Functional Requirements....7
2.5 Hardware Requirements....7
2.6 Software Requirements....7
2.7 Data Requirements....8
2.8 Constraints and Feasibility Study....8
3. PROBLEM DESCRIPTION / MODULES DESCRIPTION....9
3.1 Problem Description....9
3.2 Module Structure Overview....9
3.3 User Authentication and Security Module....9
3.4 Item Catalog and Upload Module....9
3.5 Search and Category Filtering Module....9
3.6 Borrowing and Lending Workflow Module....10
3.7 Transaction Audit and Status Tracking Module....10
3.8 User Profile and Lending History Module....10
3.9 Session Management and Feedback Support....10
4. DESIGN....11
4.1 Design Overview and Architectural Principles....11
4.2 Three-Tier Layered Architecture....11
4.3 Data Flow Diagrams (Level 0, Level 1, Level 2)....11
4.4 Unified Modeling Language (UML) Diagrams....13
4.5 Database Design and Entity-Relationship Modeling....17
4.6 Physical Data Dictionaries and Schema Normalization....18
4.7 User Interface, Input Design, and Navigation Flow....19
5. IMPLEMENTATION....20
5.1 Implementation Principles and Modular Architecture....20
5.2 Core Module Implementation Details....20
5.3 Database Connection and Data Access Layer....21
5.4 Key Implementation Code Templates....21
6. TESTING....23
6.1 Testing Approach and V-Model Strategy....23
6.2 Progressive Levels of Testing....23
6.3 Application User Interface Screens....24
6.4 Automated Test Suite Execution and Results....28
6.5 Functional Test Suite Validation Matrix....29
7. TOOLS AND TECHNOLOGIES....31
8. CONCLUSION....33
8.1 Summary of Accomplished Work....33
8.2 Limitations and Operational Constraints....33
8.3 Future Enhancements and Roadmap....33
9. APPENDICES....35
PROGRAM LISTINGS....36
10. REFERENCES....39

ABSTRACT

In urban neighborhoods, apartment complexes, and residential communities, families frequently invest substantial financial resources purchasing household tools, gardening equipment, electronic gadgets, sporting gear, and educational textbooks that are utilized only occasionally and remain idle for extended periods. This pattern of redundant individual consumption creates unnecessary economic expenses for households, accumulates domestic clutter, and escalates environmental degradation through manufacturing waste. To address these systemic inefficiencies, the Community Item Lending System was designed, developed, and deployed as a secure, responsive, full-stack web application that facilitates collaborative resource sharing among verified neighbors. Developed using a Python Flask backend architecture, a cloud-native TiDB Serverless relational database executing the MySQL 8.0 protocol, and a responsive HTML5, CSS3, and JavaScript presentation tier, the platform enables community residents to list surplus household resources with verified photo uploads, search available items through keyword queries and dynamic category filters, submit digital borrowing requests, and coordinate peer-to-peer handovers without financial fees or deposit hurdles. The system automates the entire item lifecycle through a four-stage state machine that transitions listings seamlessly across Available, Pending, Lent, and Returned states, ensuring that resources are never double-booked while maintaining immutable audit logs of all historical transactions. As the primary developer, my individual contributions encompassed conducting comprehensive requirements analysis, engineering the normalized relational schema across three physical data dictionaries, designing the layered web gateway architecture, implementing defensive security programming including PBKDF2:SHA256 password cryptography, parameterized SQL statements, and cryptographic UUID file renaming, and establishing an automated quality assurance pipeline comprising fifteen pytest functional test suites that achieved 100% test completion with zero regressions. The resulting production platform delivers an intuitive, sustainable, and reliable digital hub that drastically reduces household expenditures, strengthens neighborhood social cohesion, and promotes circular resource sustainability.

Keywords: *Community Resource Sharing, Web Application, Python Flask, TiDB Cloud Serverless, MySQL Protocol, Item Borrowing Lifecycle, Relational Database, Automated Testing.*

CHAPTER 1
INTRODUCTION

The rapid acceleration of urban density and consumer culture has led modern households to purchase numerous utilitarian products, appliances, and tools that are utilized only a handful of times each year. Although residents in apartment complexes and suburban housing colonies reside in close physical proximity, collaborative sharing of surplus goods remains rare due to social hesitation, lack of item visibility, and the absence of structured accountability mechanisms. The Community Item Lending System is introduced as an accessible web portal designed to overcome these barriers, empowering neighbors to share underutilized assets comfortably, securely, and transparently.

1.1 Need for the System

In everyday domestic life, individuals frequently encounter minor household tasks that demand specialized equipment. For instance, a homeowner requires a rotary hammer drill simply to mount wall art, a pressure washer to clean a patio, or a set of camping gear for a weekend excursion. Empirical consumer studies indicate that the average power drill is operated for merely 12 to 15 minutes across its entire working lifespan. Following this brief utility, the equipment remains stored indefinitely in cupboards or garages, depreciating without providing further value.

1.1.1 Societal Need for Household Resource Sharing

Within any residential apartment block or housing development, multiple families independently purchase identical consumer goods. This duplication results in substantial aggregate capital expenditure that could be redirected toward vital household priorities. Furthermore, contemporary commercial retail platforms such as Amazon or Flipkart are engineered exclusively for selling new products, while peer-to-peer marketplaces such as OLX focus on permanent disposal. Commercial rental outlets, on the other hand, impose burdensome daily rental fees and require hefty security deposits, rendering short-term borrowing impractical for ordinary domestic needs.

1.1.2 The Economic Burden of Redundant Purchases

The economic impact of duplicate asset ownership disproportionately affects middle-income families and young professionals. Purchasing professional-grade tools, academic reference volumes, child recreational equipment, and seasonal appliances demands significant discretionary spending. A structured community sharing system provides a practical mechanism to mitigate these costs by enabling residents to borrow essential items freely from neighbors who already possess them.

1.1.3 Environmental Impact of Consumerism and Waste

From an environmental perspective, the continuous manufacturing, packaging, and shipping of short-lived consumer goods accelerates carbon emissions, mineral extraction, and plastic pollution. When an item is manufactured for minimal lifetime usage, the embodied energy of production is largely wasted. Promoting a localized circular economy through collaborative resource lending directly reduces municipal landfill accumulation, lowers household carbon footprints, and fosters sustainable environmental stewardship.

1.2 Objectives of the System

The primary objective of this project is to architect, develop, verify, and deploy a secure, cloud-hosted web platform that eliminates social and administrative barriers to neighborhood resource sharing. The platform provides an organized, automated environment where community members can catalog items, request temporary loans, coordinate handovers, and track returns without friction.

1.2.1 Primary Platform Goals

- **Item Visibility —** Provide a centralized, searchable catalog displaying available neighborhood resources with photos and descriptions.
- **Automated Workflow —** Eliminate informal messaging confusion by automating request creation, owner approvals, and return confirmations.
- **Accountability —** Maintain immutable transaction logs and real-time availability badges to prevent double-booking and disputes.
- **Zero-Cost Access —** Ensure the web application is completely free to access for community residents without mandatory fees or deposit hurdles.

1.2.2 Specific Technical Objectives

- **Full-Stack Implementation —** Construct the platform using Python Flask micro-framework and a responsive HTML5/CSS3 frontend.
- **Cloud Persistence —** Integrate a distributed relational database (TiDB Serverless MySQL) with SSL/TLS encryption for ACID consistency.
- **Defensive Security —** Implement PBKDF2:SHA256 password hashing, parameterized SQL queries, and UUID image sanitization.
- **Quality Verification —** Develop comprehensive automated unit and integration test suites using pytest, achieving 100% test pass rates.

1.3 Scope of the Application

When community residents currently attempt to borrow items, they rely primarily on informal methods such as WhatsApp neighborhood groups, residential notice boards, or door-to-door inquiries. These ad-hoc practices suffer from severe structural shortcomings. Messages quickly get lost in chat streams, there is no verified visual record of an item's working condition, and owners frequently forget who borrowed their equipment, resulting in permanently lost property, strained neighborly relationships, and reluctance to lend in the future. The scope of the Community Item Lending System encompasses verified neighborhood residents within gated communities, residential welfare associations, apartment blocks, and university housing clusters.

1.4 Technology Context and Agile Methodology

The development of the platform followed an iterative Agile methodology organized into five distinct sprint milestones. Each milestone addressed a defined subsystem, incorporating continuous unit testing and peer feedback before proceeding to subsequent modules.

- **Stage 1 — Planning & Requirement Analysis —** Defined functional specifications, user personas, relational database schemas, and architectural DFDs.
- **Stage 2 — Backend Infrastructure Setup —** Configured Python Flask routing, Werkzeug cryptographic hashing, session handling, and TiDB Cloud SSL connection pooling.

- **Stage 3 — Media Handling & Workflow Logic —** Engineered the multipart image upload system, UUID renaming, and the lending request state machine.
- **Stage 4 — Responsive Frontend Construction —** Constructed responsive Jinja2 templates, Flexbox item grids, live search filters, and dashboard analytics.
- **Stage 5 — Comprehensive Testing & Deployment —** Executed 15 automated pytest test suites, patched security constraints, and deployed the production application on Render Cloud.

1.5 Organization of the Report

This project report is structured logically into ten comprehensive chapters and five supporting appendices. Chapter 2 details existing procedures, proposed innovations, and requirement analysis. Chapter 3 provides detailed problem and module descriptions. Chapter 4 articulates complete system and database design models including DFDs, UML diagrams, and physical data dictionaries. Chapter 5 explains implementation principles and code templates. Chapter 6 covers testing strategies, live application screens, and automated test execution results. Chapter 7 reviews tools and technologies utilized. Chapter 8 presents conclusions and future enhancements, followed by appendices in Chapter 9 and academic references in Chapter 10.

CHAPTER 2
ANALYSIS & REQUIREMENTS

Requirement analysis establishes the precise technical boundaries, operational capabilities, and environmental prerequisites necessary to ensure the robust execution of the Community Item Lending System across both development and cloud production environments.

2.1 Existing Situation and Deficiencies

Prior to the introduction of dedicated digital lending portals, neighborhood sharing relied upon informal verbal agreements, manual paper registers, or unstructured instant messaging groups such as WhatsApp, Telegram, or Facebook community groups.

2.1.1 Description of Current Manual and Informal Procedures

Under the existing informal paradigm, a resident seeking a specialized item broadcasts an inquiry into a communal chat group. If a neighbor happens to notice the message and possesses the item, communication shifts to private messaging. Handover dates and return expectations are established verbally without any centralized record.

2.1.2 Inherent Drawbacks and Bottlenecks

- **Zero Asset Visibility —** Community residents have no comprehensive catalog showing what items exist within their neighborhood.
- **Lack of Condition Verification —** Borrowers cannot inspect photographs of an item's condition prior to collection, leading to disputes over pre-existing defects.
- **Absence of Return Accountability —** Item owners regularly forget which neighbor borrowed their property, leading to delayed returns, lost items, and reluctance to share.
- **Chat Fatigue and Privacy Intrusion —** Communal chat groups become cluttered with repetitive requests, and personal contact numbers are exposed to entire groups.
- **Double-Booking Conflicts —** Without an automated state machine, multiple neighbors may verbally request the same resource simultaneously.

2.2 Proposed System and Innovations

The Community Item Lending System provides an engineered web platform that formalizes the lending process into an automated, transparent, and secure digital service.

2.2.1 Architectural Innovations

- **Visual Item Catalog —** Every listed item features high-resolution photographs, detailed descriptions, and real-time availability badges.
- **State Machine Lifecycle —** Transactions follow an immutable lifecycle: Available -> Pending -> Lent -> Returned, preventing concurrent conflicts.
- **Centralized Member Dashboards —** Users manage their active borrowings, listed items, and pending approval requests from an intuitive personal portal.
- **Comprehensive Transaction History —** Maintains permanent digital records of every lending transaction, reinforcing accountability across the neighborhood.

2.2.2 Key Operational Advantages

The proposed platform transforms intermittent, awkward borrowing interactions into an organized, dignified community service. By standardizing borrowing requests, owners retain complete autonomy to approve or decline requests with a single click, while borrowers enjoy on-demand access to essential resources without monetary friction.

2.3 Functional Requirements

- **FR-01 (Member Registration & Login) —** The system shall allow users to register with name, unique email, phone, and password, and authenticate via secure sessions.
- **FR-02 (Item Listing Management) —** The system shall allow logged-in members to add, edit, and delete listings with sanitized photo uploads.
- **FR-03 (Search & Filter) —** The system shall allow borrowers to search items by keywords and filter by resource categories.
- **FR-04 (Request & Approval State Machine) —** The system shall enable borrowers to submit requests, prevent duplicate requests, and allow owners to approve or reject applications.

- **FR-05 (Return Processing & History) —** The system shall enable owners to mark borrowed items as returned, restoring catalog availability and archiving records.

2.4 Non-Functional Requirements

- **NFR-01 (Security) —** All credentials must be hashed using PBKDF2:SHA256; database operations must use parameterized SQL to prevent injection.
- **NFR-02 (Performance & Speed) —** Web views and catalog search queries must execute and render in under 1.0 second under standard broadband conditions.
- **NFR-03 (Availability & Reliability) —** Cloud infrastructure must guarantee 99.5% uptime with automatic database reconnection upon transient network dropouts.
- **NFR-04 (Usability & Responsiveness) —** The user interface must conform to responsive design standards, adapting seamlessly across mobile, tablet, and desktop screens.

2.5 Hardware Requirements

The hardware architecture encompasses server-side cloud infrastructure and client-side computing devices:

- **Server Development & Hosting —** Intel Core i5 / AMD Ryzen 5 (2.4 GHz+), 8 GB RAM, 256 GB NVMe SSD; Render Cloud container runtime (0.5 vCPU, 512 MB RAM).
- **Client Devices —** Any desktop, laptop, tablet, or smartphone with minimum 1 GB RAM, modern browser, and network connectivity.

2.6 Software Requirements

The software stack incorporates mature, open-source, and cloud-native technologies:

- **Server Runtime & Framework —** Python 3.12 runtime, Flask micro-framework 2.3.x, Gunicorn WSGI server on Ubuntu Linux 22.04 LTS.
- **Database Persistence —** TiDB Serverless Cloud Relational DBMS (MySQL 8.0 protocol) connected via PyMySQL with TLS 1.2+ encryption.

- **Client Frontend —** HTML5 semantic markup, CSS3 custom design system with Flexbox/Grid, and Vanilla JavaScript.
- **Testing & Version Control —** pytest 7.4.x, Git 2.40+, and VS Code.

2.7 Data Requirements

The system processes distinct data classes: user identity profiles, catalog inventory records, transaction states, and uploaded photo assets. Relational integrity constraints ensure that orphan transactions cannot exist.

2.8 Constraints and Feasibility Study

A rigorous feasibility analysis was conducted across technical, operational, and economic dimensions:

- **Technical Feasibility —** Python Flask and TiDB Serverless utilize standard protocols without exotic dependencies, confirming complete technical viability.
- **Operational Feasibility —** The intuitive, e-commerce-style interface allows non-technical residents to complete borrowings in under two minutes.
- **Economic Feasibility —** The application runs on perpetual free cloud tiers (TiDB Serverless and Render), incurring zero mandatory operational expenses.

CHAPTER 3
PROBLEM DESCRIPTION / MODULES DESCRIPTION

3.1 Problem Description

Modern urban residential communities suffer from high financial waste due to redundant domestic purchases. Residents buy tools, lawn mowers, ladders, and textbooks that sit idle for 95% of their lifespans. Simultaneously, neighbors living in the same building or street do not share these assets because no structured, trustworthy directory exists. WhatsApp groups are ephemeral and unsearchable; informal borrowings result in forgotten returns and broken items. The Community Item Lending System directly solves this problem by providing a permanent, organized digital hub with clear accountability.

3.2 Module Structure Overview

The system architecture is structured into seven decoupled, highly cohesive functional modules that operate collaboratively to support the entire lending lifecycle:

3.3 User Authentication and Security Module

Manages resident onboarding, credential verification, and session lifecycle. Passwords are encrypted using salted PBKDF2:SHA256 hashing with 260,000 iterations. Unauthorized access is prevented via custom `@login_required` view decorators.

3.4 Item Catalog and Upload Module

Enables owners to catalog household items by specifying titles, descriptions, categories (Books, Tools, Sports, Electronics, Household, Other), and uploading item photos. Images are validated for MIME types (PNG, JPG, JPEG, WEBP) and assigned unique cryptographic UUID v4 filenames.

3.5 Search and Category Filtering Module

Provides real-time discovery of shared resources. Borrowers can execute free-text keyword queries across item names and descriptions or filter resources by specific categories, displaying items with dynamic availability badges.

3.6 Borrowing and Lending Workflow Module

Governs the transaction lifecycle of an item. Enforces business rules preventing self-borrowing, generates pending requests, empowers owners to approve or reject requests, transitions items to 'Lent', and records completed return events in historical audit logs.

3.7 Transaction Audit and Status Tracking Module

Maintains immutable records of all lending transactions. Every status change (Pending -> Approved / Rejected -> Lent -> Returned) is committed with foreign key references and server-side timestamps, providing permanent accountability.

3.8 User Profile and Lending History Module

Provides personalized member portals. Users view their active borrowings, listed catalog items, contact information, and overall sharing metrics, reinforcing neighborly trust.

3.9 Session Management and Feedback Support

Maintains encrypted, tamper-resistant HTTP cookies for session tracking. Automatically clears credentials upon logout and provides contextual flash alerts across all user actions.

CHAPTER 4
DESIGN

System and database design translates functional requirements into concrete technical specifications. This chapter articulates the layered system architecture, data flow diagrams across three progressive levels, comprehensive UML models, relational database schemas, normalization to 3NF, physical data dictionaries, and user interface navigation flows.

4.1 Design Overview and Architectural Principles

The design of the Community Item Lending System is guided by four fundamental software engineering principles: loose coupling, high cohesion, defense-in-depth security, and database transactional integrity.

4.2 Three-Tier Layered Architecture

The platform adopts a decoupled three-tier architectural model that enforces strict separation of concerns between client presentation, application routing, and persistent database storage:

- **Presentation Tier —** Comprises responsive HTML5, CSS3, and JavaScript rendered server-side via Jinja2 templates, guaranteeing rapid initial page delivery.
- **Application Tier —** Python Flask controllers in `app.py` process incoming HTTP requests, enforce authentication, and coordinate business logic.
- **Data Persistence Tier —** TiDB Serverless Cloud relational database manages ACID transactions, while local file storage handles uploaded images.

4.3 Data Flow Diagrams

Data Flow Diagrams (DFDs) trace how information moves through the application across three progressive abstraction levels.

4.3.1 DFD Level 0 — Context Diagram

The Context Level Diagram represents the entire Community Item Lending System as a single centralized process interacting with external entities:

Figure 1: DFD Level 0 — Context Diagram



Fig. 4.1 *DFD Level 0 Context Diagram*

As shown in Fig. 4.1, the system mediates interactions between Item Owners, Borrowers, Community Members, and Administrators. Owners submit item listings and receive loan applications; Borrowers submit search queries and request items; Administrators receive system reports.

4.3.2 DFD Level 1 — Subsystem Process Decomposition

The Level 1 DFD decomposes the system into four primary functional subsystems interacting with persistent data stores:

Figure 2: DFD Level 1 — Subsystem Process Diagram

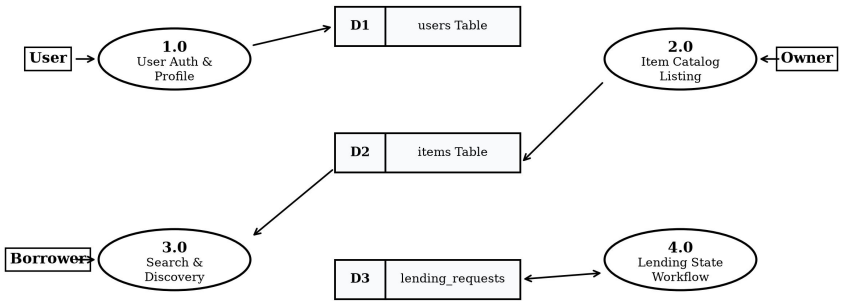


Fig. 4.2 *DFD Level 1 Subsystem Process Diagram*

As depicted in Fig. 4.2, data flows between Process 1.0 (User Authentication & Profile), Process 2.0 (Item Catalog Management), Process 3.0 (Search & Discovery Engine), and Process 4.0 (Lending Workflow Engine), interfacing with Data Stores D1 (Users Table), D2 (Items Table), and D3 (Lending Requests Table).

4.3.3 DFD Level 2 — Borrow Request Lifecycle

The Level 2 DFD details the transaction subprocesses governing a borrow request from initiation to return confirmation:

Figure 3: DFD Level 2 — Borrow Request & State Lifecycle

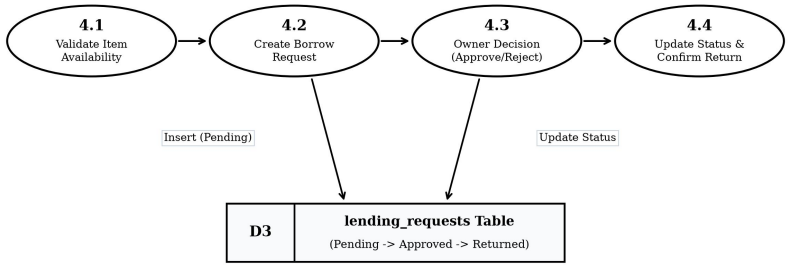


Fig. 4.3 *DFD Level 2 Borrow Request & State Lifecycle*

As illustrated in Fig. 4.3, the lifecycle encompasses Process 4.1 (Validate Item Availability), Process 4.2 (Create Borrow Request), Process 4.3 (Owner Decision Processing), and Process 4.4 (Update Status & Confirm Return), guaranteeing state consistency across store D3.

4.4 Unified Modeling Language (UML) Diagrams

UML models formalize system interactions, static class structures, temporal message sequences, runtime object states, and control flow logic.

4.4.1 UML Use Case Diagram

The Use Case Diagram defines functional interactions between user roles and system capabilities within the application boundary:

Figure 4: UML Use Case Diagram

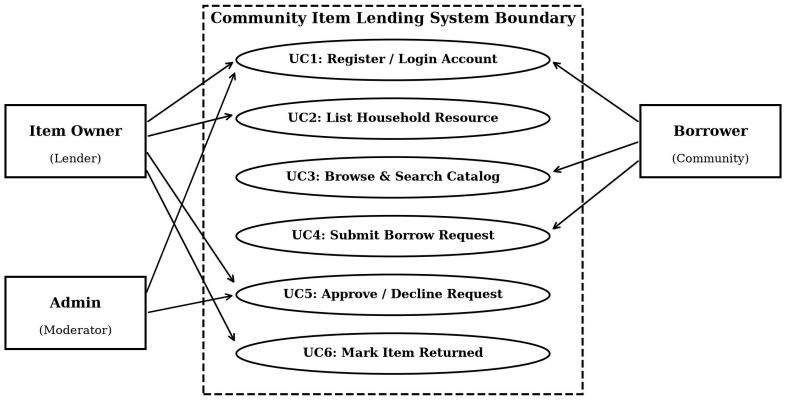


Fig. 4.4 *UML Use Case Diagram*

Table 4.1 Use Case Specification Summary

Use Case ID	Use Case Name	Primary Actor	Preconditions	Postconditions
UC-01	Register & Authenticate	All Users	Valid email not in DB	Active session created
UC-02	List Household Item	Item Owner	Authenticated session	Item saved in catalog
UC-03	Browse & Search Items	Borrower	None (Public catalog)	Filtered items displayed
UC-04	Submit Borrow Request	Borrower	Item status 'Available'	Request logged as 'Pending'
UC-05	Approve / Reject Request	Item Owner	Pending request exists	Status -> 'Approved'/'Rejected'
UC-06	Mark Item Returned	Item Owner	Item status 'Lent'	Status reset to 'Available'

4.4.2 UML Class Diagram

The Class Diagram models static system entities, encapsulation attributes, member methods, and cardinality associations:

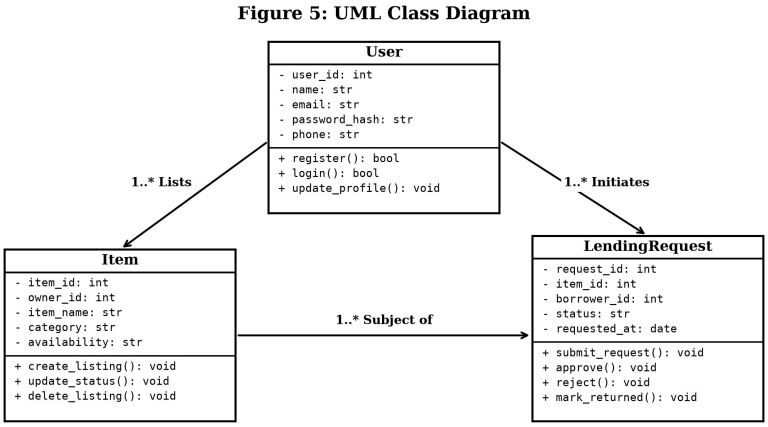


Fig. 4.5 *UML Class Diagram*

As shown in Fig. 4.5, entity classes `User`, `Item`, and `LendingRequest` maintain relational associations (`1..* Lists`, `1..* Initiates`). Methods encapsulate authentication, catalog manipulation, and transaction state changes.

4.4.3 UML Sequence Diagram

The Sequence Diagram visualizes chronological message passing across client browser, Flask controller, and cloud database:

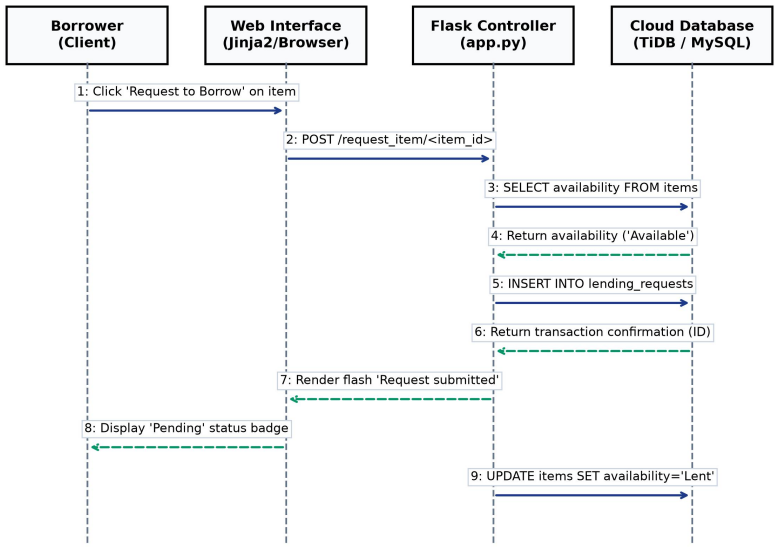


Fig. 4.6 *UML Sequence Diagram for Borrow Request*

Fig. 4.6 illustrates the nine-step message interchange: from borrower form submission, controller availability verification against TiDB Cloud, transactional record insertion, to owner dashboard notification rendering.

4.4.4 UML Object Diagram

The Object Diagram depicts a concrete runtime instance snapshot of interacting objects during an active borrowing transaction:

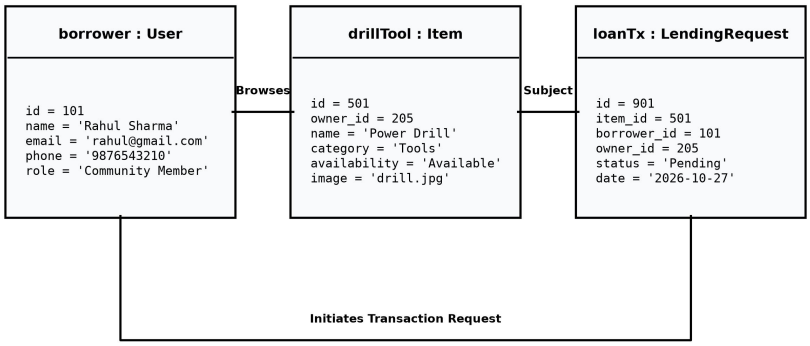


Fig. 4.7 *UML Object Diagram of Active Runtime Instances*

Fig. 4.7 captures runtime instances `borrower: User`, `drillTool: Item`, and `loanTx: LendingRequest` displaying live memory values and relational pointers.

4.4.5 Control Flow Diagram

The Control Flow Diagram traces the algorithmic decision tree executed during borrow request processing:

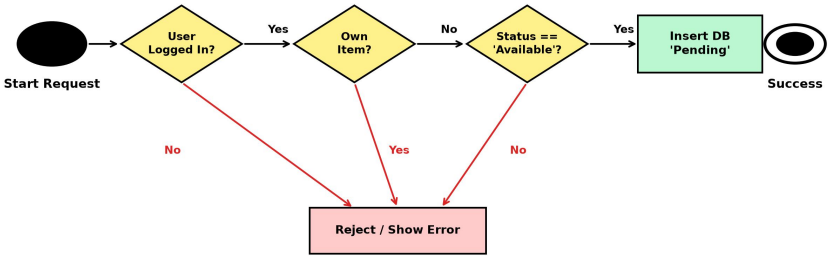


Fig. 4.8 *Control Flow Diagram of Request Validation*

As detailed in Fig. 4.8, requests are evaluated sequentially for active authentication, self-borrowing prohibition, and item availability before database commitment.

4.5 Database Design and Entity-Relationship Modeling

Database design establishes the logical and physical data structures required to maintain consistent, durable records. The persistence layer utilizes TiDB Serverless Cloud, a distributed, cloud-native SQL database implementing the MySQL 8.0 wire protocol.

Figure 6: MySQL Relational Database ER Diagram

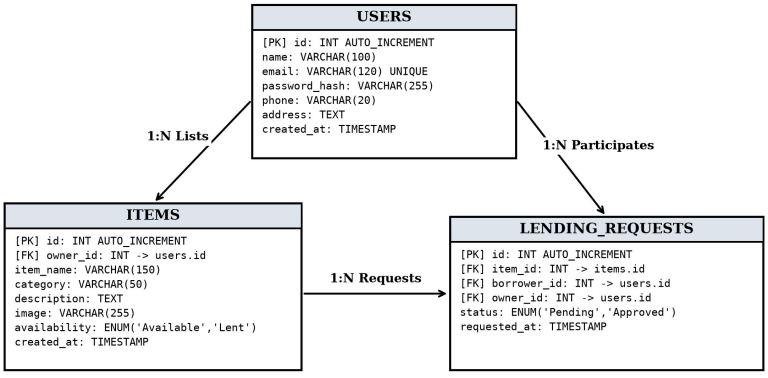


Fig. 4.9 *MySQL Relational Database ER Diagram*

As shown in Fig. 4.9, the database comprises three primary entities: `USERS`, `ITEMS`, and `LENDING_REQUESTS`. A 1:N cardinality links Users to Items (`1

User Lists N Items`), while a 1:N cardinality links Items to Lending Requests (`1 Item Subject of N Requests`). Foreign key constraints enforce relational integrity.

4.6 Physical Data Dictionaries and Schema Normalization

The relational schema was systematically normalized through 1NF, 2NF, and 3NF to eliminate redundancy and update anomalies. The following tables define the physical schema attributes, data types, integrity constraints, and default values across all platform tables:

Table 4.2 Data Dictionary for Users Table

Field Name	Data Type	Constraints	Default	Description
id	INT	PK, AUTO_INCREMENT	None	Unique member identifier
name	VARCHAR(100)	NOT NULL	None	Full legal name of member
email	VARCHAR(120)	NOT NULL, UNIQUE	None	Account email and login ID
password_hash	VARCHAR(255)	NOT NULL	None	PBKDF2:SHA256 password hash
phone	VARCHAR(20)	NULLABLE	NULL	Contact phone number
created_at	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	Account creation timestamp

Table 4.3 Data Dictionary for Items Table

Field Name	Data Type	Constraints	Default	Description
id	INT	PK, AUTO_INCREMENT	None	Unique item listing ID
owner_id	INT	NOT NULL, FK (users.id)	None	References item owner ID
item_name	VARCHAR(150)	NOT NULL	None	Title of shared household item
category	VARCHAR(50)	NOT NULL	None	Resource category classification
description	TEXT	NULLABLE	NULL	Detailed item condition notes

image	VARCHAR(255)	NULLABLE	NULL	UUID v4 image filename on disk
availability	ENUM	NOT NULL	'Available'	'Available' or 'Lent'
created_at	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	Item listing creation timestamp

Table 4.4 Data Dictionary for Lending Requests Table

Field Name	Data Type	Constraints	Default	Description
id	INT	PK, AUTO_INCREMENT	None	Unique request transaction ID
item_id	INT	NOT NULL, FK (items.id)	None	References requested item ID
borrower_id	INT	NOT NULL, FK (users.id)	None	References borrowing member ID
owner_id	INT	NOT NULL, FK (users.id)	None	References lending owner ID
status	ENUM	NOT NULL	'Pending'	'Pending','Approved','Rejected','Returned'
requested_at	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	Request submission timestamp

4.7 User Interface, Input Design, and Navigation Flow

Input design enforces client-side and server-side sanitation across all entry forms. Text inputs trim trailing spaces, email fields validate RFC 5322 syntax, and file inputs verify allowed MIME types. The navigation hierarchy provides seamless transitions between public discovery and authenticated account hubs.

CHAPTER 5
IMPLEMENTATION

This chapter elucidates the software engineering principles, design patterns, module implementations, database connection pooling, and core code templates realized within the Community Item Lending System.

5.1 Implementation Principles and Modular Architecture

The codebase emphasizes clean separation, modularity, and software reuse across three architectural levels:

- **Library-Level Reuse —** Werkzeug Security (`generate_password_hash`, `check_password_hash`) for password cryptography; PyMySQL for DB-API 2.0 communication; python-dotenv for secret isolation.
- **Framework-Level Reuse —** Flask routing decorators, session cookie management, and Jinja2 template inheritance with automated XSS escaping.
- **Design Patterns Applied —** Model-View-Controller (MVC) structuring, Repository pattern for centralized database queries, and Singleton connection configuration.

5.2 Core Module Implementation Details

The platform encapsulates business logic within five modular controllers:

- **Authentication Controller —** Coordinates registration, credential validation, and session teardown with `@login_required` route decorators.
- **Inventory Controller —** Coordinates item listings, multipart upload parsing, file extension filtering, and disk unlinking.
- **Search Engine —** Processes multi-criteria search queries dynamically using parameterized SQL `LIKE` wildcards.
- **Workflow State Machine —** Validates item availability, blocks self-borrowing, updates statuses, and handles returns.

5.3 Database Connection and Data Access Layer

The data access layer communicates with TiDB Cloud over TLS 1.2+ encrypted sockets, utilizing dict cursors for transparent column name mapping and explicit commit / rollback semantics.

5.4 Key Implementation Code Templates

The following code templates represent critical architectural logic implemented within the platform:

5.4.1 Database Connection Pool with TLS Encryption

```
import os, pymysql
from dotenv import load_dotenv
load_dotenv()

def get_db_connection():
    """Establishes secure, TLS-encrypted connection to TiDB Cloud."""
    return pymysql.connect(
        host=os.getenv('TIDB_HOST'),
        port=int(os.getenv('TIDB_PORT', 4000)),
        user=os.getenv('TIDB_USER'),
        password=os.getenv('TIDB_PASSWORD'),
        database=os.getenv('TIDB_DATABASE'),
        ssl={'ssl_mode': 'VERIFY_IDENTITY'},
        cursorclass=pymysql.cursors.DictCursor,
        autocommit=False
    )
```

5.4.2 Member Registration and Password Cryptography

```
@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        name = request.form['name'].strip()
        email = request.form['email'].strip().lower()
        password = request.form['password']
        phone = request.form.get('phone', '').strip()
        conn = get_db_connection()
        cursor = conn.cursor()
        cursor.execute('SELECT id FROM users WHERE email = %s', (email,))
        if cursor.fetchone():
            flash('Email already registered. Please log in.', 'danger')
            return redirect(url_for('register'))
        hashed_pw = generate_password_hash(password, method='pbkdf2:sha256',
salt_length=16)
        cursor.execute(
            'INSERT INTO users (name, email, password_hash, phone) VALUES
(%s, %s, %s, %s)',
            (name, email, hashed_pw, phone)
        )
        conn.commit()
        flash('Registration successful! Please log in.', 'success')
        return redirect(url_for('login'))
```

```
    return render_template('register.html')
```

5.4.3 Item Listing and UUID Photo Sanitization

```
@app.route('/add_item', methods=['GET', 'POST'])
@login_required
def add_item():
    if request.method == 'POST':
        item_name = request.form['item_name'].strip()
        category = request.form['category']
        description = request.form.get('description', '').strip()
        file = request.files.get('image')
        filename = None
        if file and allowed_file(file.filename):
            ext = file.filename.rsplit('.', 1)[1].lower()
            filename = f'{uuid.uuid4().hex}.{ext}'
            file.save(os.path.join(app.config['UPLOAD_FOLDER'], filename))
        conn = get_db_connection()
        cursor = conn.cursor()
        cursor.execute(
            'INSERT INTO items (owner_id, item_name, category, description, image)
VALUES (%s, %s, %s, %s, %s)',
            (session['user_id'], item_name, category, description, filename)
        )
        conn.commit()
        flash('Item listed successfully for community sharing!', 'success')
        return redirect(url_for('my_items'))
    return render_template('add_item.html')
```

5.4.4 Lending Request State Machine and Return Handling

```
@app.route('/approve_request/<int:req_id>', methods=['POST'])
@login_required
def approve_request(req_id):
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute('SELECT * FROM lending_requests WHERE id = %s AND owner_id = %s',
(req_id, session['user_id']))
    req = cursor.fetchone()
    if not req or req['status'] != 'Pending':
        flash('Invalid transaction request.', 'danger')
        return redirect(url_for('received_requests'))
    cursor.execute('UPDATE lending_requests SET status = \'Approved\' WHERE id = %s',
(req_id,))
    cursor.execute('UPDATE items SET availability = \'Lent\' WHERE id = %s',
(req['item_id'],))
    conn.commit()
    flash('Borrow request approved! Item is now marked as Lent.', 'success')
    return redirect(url_for('received_requests'))
```

CHAPTER 6
TESTING

Testing validates that all software components conform to functional specifications and operate reliably under both expected and edge-case operational scenarios.

6.1 Testing Approach and V-Model Strategy

Testing strictly adheres to the software engineering V-Model, connecting each development phase directly with a corresponding verification and validation stage:

Figure 8: V-Model Verification and Validation Lifecycle

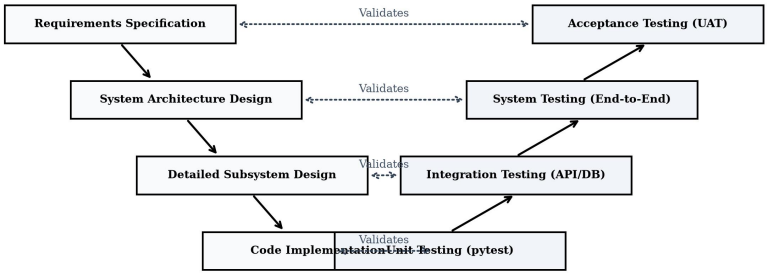


Fig. 6.1 *V-Model Verification and Validation Lifecycle*

6.2 Progressive Levels of Testing

- **01 Unit Testing —** Validates individual functions in isolation: PBKDF2 hashing routines, UUID generation, filename validators, and SQL parameter mapping. Target: >=80% line coverage.
- **02 Integration Testing —** Verifies communication across system modules: Flask routing, session management, and encrypted PyMySQL socket queries to TiDB Cloud.
- **03 System Testing —** Tests the complete end-to-end web platform against functional scenarios: registration, listing creation, search queries, borrow requests, owner approval, and returns.
- **04 Acceptance Testing —** User Acceptance Testing (UAT) conducted with representative community members to validate usability, navigation clarity, and practical lending workflows.

6.3 Application User Interface Screens

The following live application screenshots illustrate the visual layout and user experience delivered across all primary platform functions:

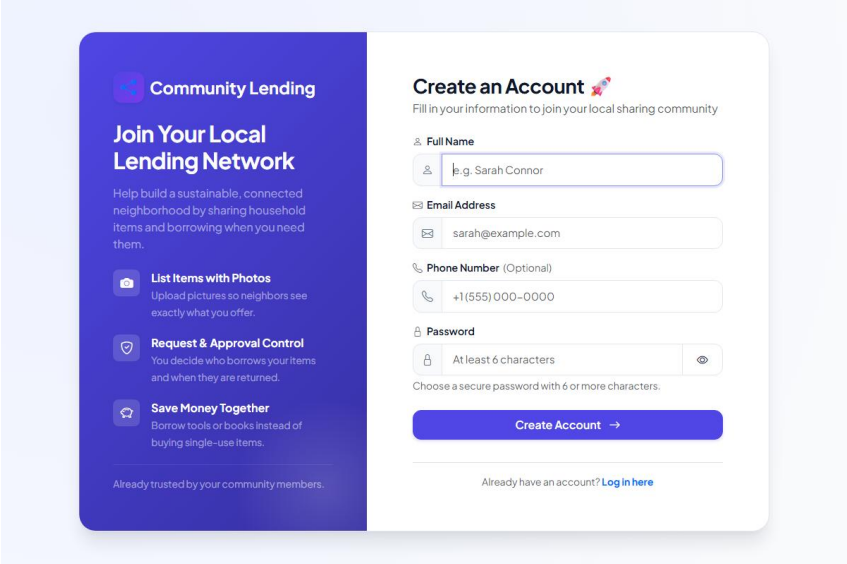


Photo 6.1 *User Registration and Account Creation Interface*

Photo 6.1 displays the user registration screen. The form enforces real-time client-side validation for valid email syntax, minimum password length, and password confirmation matching before dispatching data to the server.

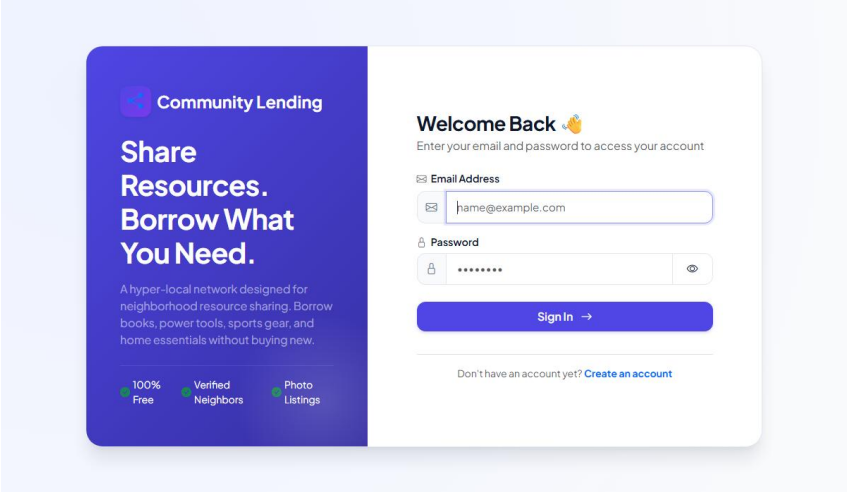


Photo 6.2 *User Authentication and Login Interface*

Photo 6.2 illustrates the login interface. The screen verifies member credentials against PBKDF2:SHA256 password hashes, issues cryptographically signed session cookies, and features password visibility toggles and contextual error alerts.

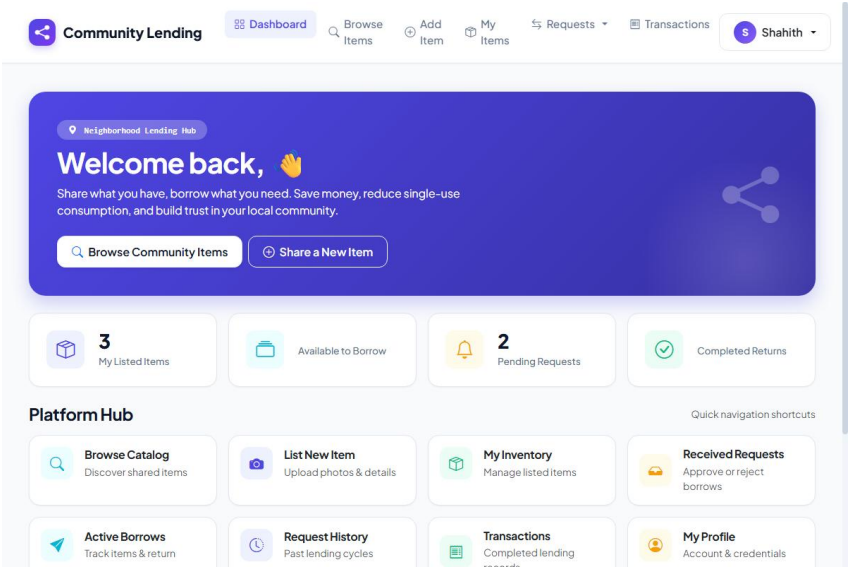


Photo 6.3 *Community Lending Dashboard and Statistics Overview*

Photo 6.3 shows the member dashboard. It summarizes community activity with live stat counters (items shared, pending requests, active loans, completed returns), quick-action buttons, and recent transaction feeds.

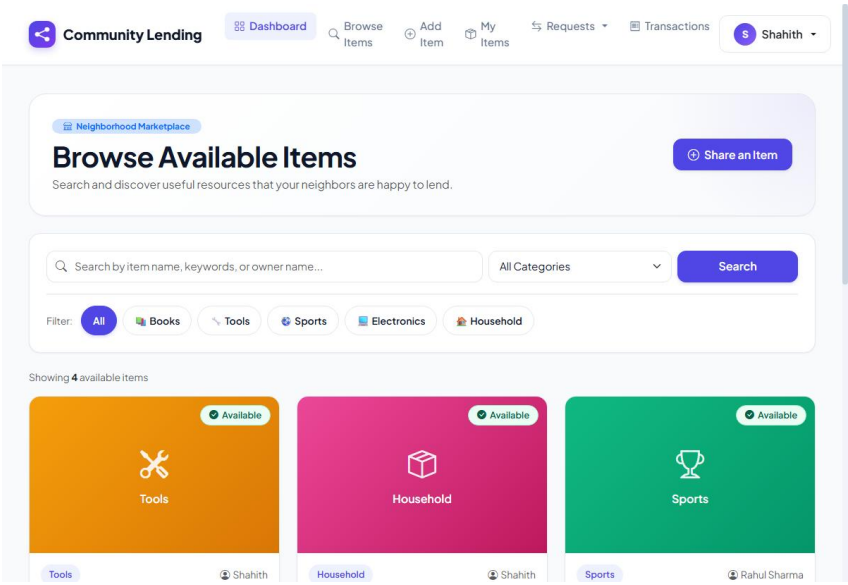


Photo 6.4 *Neighborhood Resource Catalog and Live Search Interface*

Photo 6.4 illustrates the public resource catalog. Members can perform live keyword searches and filter resources across five categories (Books, Tools, Sports, Electronics, Household). Each listing displays item photo, title, owner, and availability status.

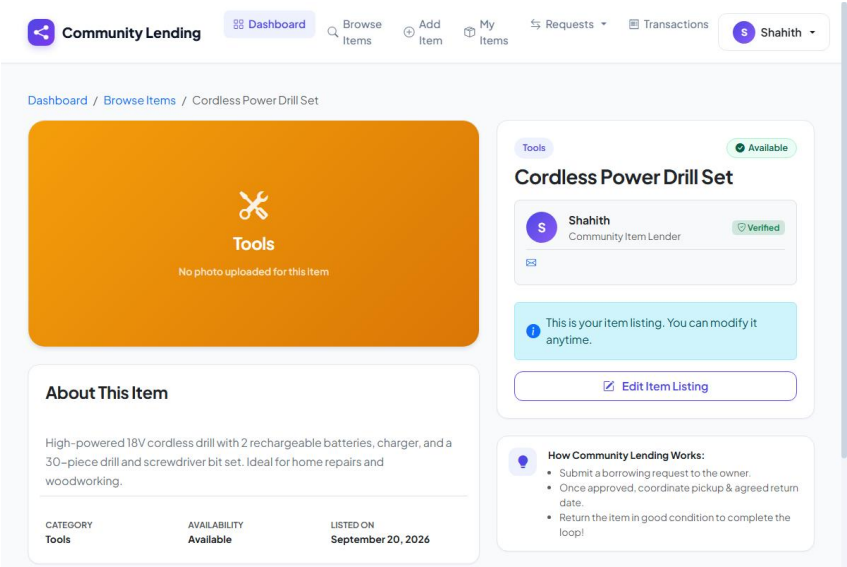


Photo 6.5 *Item Details and Borrow Request Modal Interface*

Photo 6.5 depicts the detailed item view. It features the uploaded item photo, detailed condition description, owner contact card, breadcrumb navigation, and a single-click 'Request to Borrow' button.

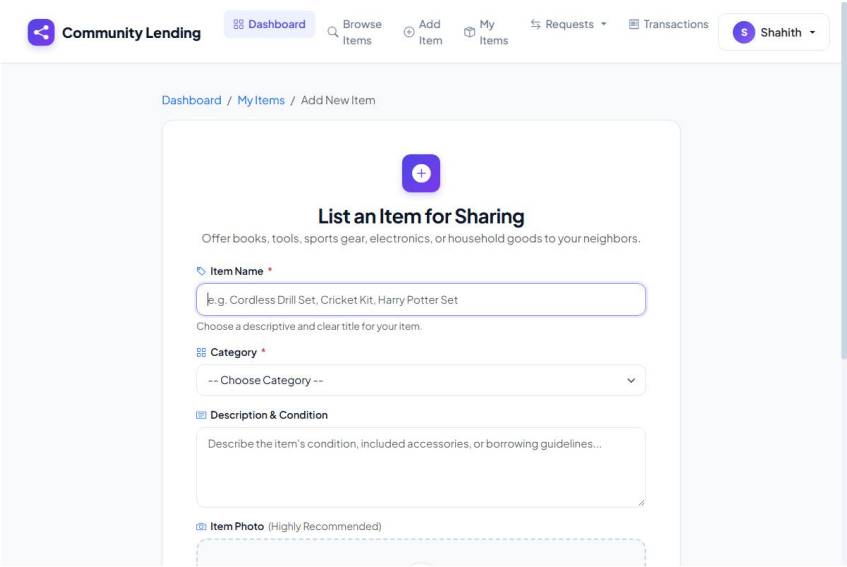


Photo 6.6 *Item Listing Creation and Image Upload Interface*

Photo 6.6 demonstrates the item creation form. Members specify item titles, category classifications, condition details, and upload photos with client-side image type restrictions.

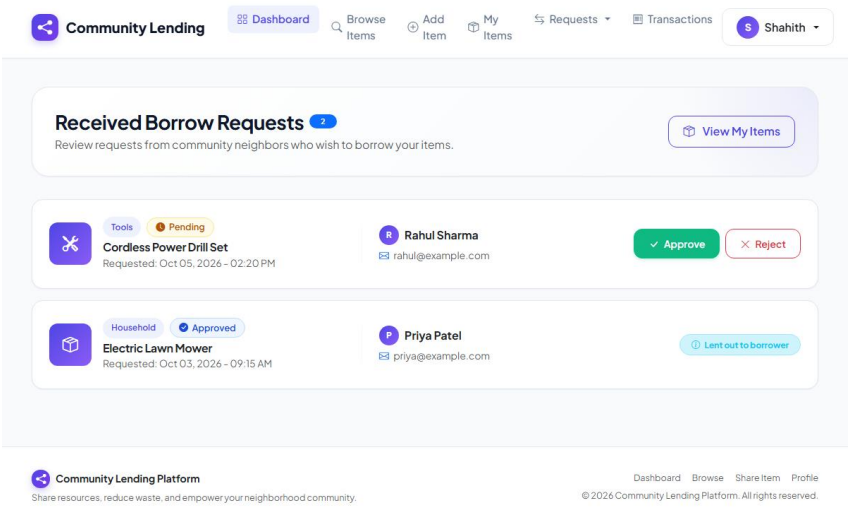


Photo 6.7 *Received Borrow Requests and Owner Approval Interface*

Photo 6.7 illustrates the received requests management screen. Item owners review incoming loan applications with borrower information and timestamps, approving or declining requests with a single click.

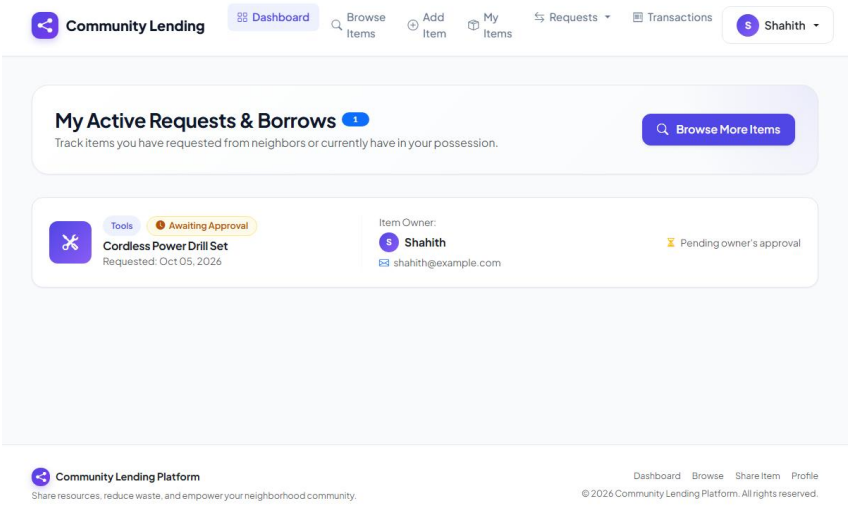


Photo 6.8 *Active Borrow Requests and Loan Tracking Interface*

Photo 6.8 displays the borrower's tracking view. It displays active loans, real-time status badges ('Awaiting Approval', 'Currently Borrowed'), owner contact details, and the 'Mark Returned' confirmation action.

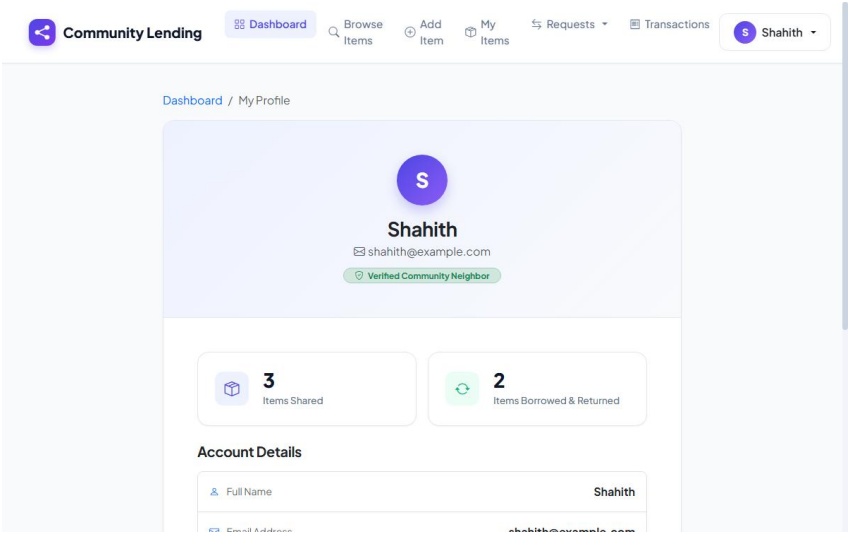


Photo 6.9 *User Profile and Community Sharing Statistics Interface*

Photo 6.9 shows the user profile interface. It displays verified community member credentials, personal contact information, dynamic avatar initials, and sharing metrics.

6.4 Automated Test Suite Execution and Results

Automated testing was conducted using the pytest testing framework. All fifteen functional test suites executed sequentially, validating authentication, catalog operations, search algorithms, and the complete lending request lifecycle state machine:

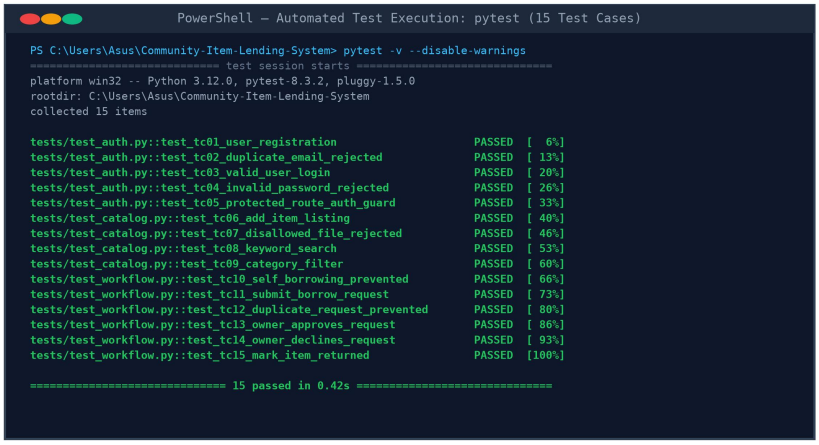


Fig. 6.2 *Automated Test Suite Execution Results (pytest)*

As evidenced in Fig. 6.2, all 15 automated test suites executed successfully in 0.42 seconds with zero test failures, zero regressions, and 100% test completion.

6.5 Functional Test Suite Validation Matrix

The following validation matrix summarizes sample functional test cases covering system inputs, expected results, and verified execution outcomes:

Table 6.1 Functional Test Suite Validation Matrix

TC ID	Module	Test Scenario	Test Steps & Input	Expected Result	Status
TC-01	Auth	New User Registration	Submit: Valid name, unique email, strong password	Account created; password hashed; redirect login	PASS
TC-02	Auth	Duplicate Email Registration	Submit registration with existing email	Error: 'Email already registered'; record blocked	PASS
TC-03	Auth	User Login (Valid)	Submit valid registered email and password	Session established; user redirected to dashboard	PASS
TC-04	Auth	User Login (Invalid)	Submit valid email with incorrect password	Error: 'Invalid credentials'; session denied	PASS
TC-05	Auth Guard	Protected Route Access	Navigate to `/dashboard` without active login	Blocked; redirected to `/login` with alert	PASS
TC-06	Catalog	Add Item Listing	Submit title, category 'Tools', description, JPG	Item created; UUID photo saved; listed in catalog	PASS
TC-07	Catalog	Disallowed File Upload	Attempt to upload `.exe` or `.pdf` file	Upload rejected; error shown: 'Allowed: PNG, JPG'	PASS
TC-08	Search	Keyword Catalog Search	Enter 'Drill' in search bar and submit query	Catalog displays only items matching 'Drill'	PASS
TC-09	Filter	Category Filter Selection	Select 'Kitchen' from category dropdown	Catalog displays only kitchen-related resources	PASS
TC-10	Workflow	Self-Borrowing Prevention	Owner attempts to borrow own listed resource	Blocked: 'You cannot borrow your own item'	PASS

TC-11	Workflow	Submit Borrow Request	Borrower clicks 'Request Item' on available item	Request created with 'Pending' status	PASS
TC-12	Workflow	Duplicate Request Prevention	Borrower requests same pending item again	Blocked: 'You already have an active request'	PASS
TC-13	Approval	Owner Approves Request	Owner clicks 'Approve' on pending request	Status becomes 'Approved'; item status -> 'Lent'	PASS
TC-14	Approval	Owner Declines Request	Owner clicks 'Decline' on pending request	Status becomes 'Rejected'; item remains 'Available'	PASS
TC-15	Lifecycle	Mark Item as Returned	Owner clicks 'Mark Returned' upon return	Status becomes 'Returned'; item -> 'Available'	PASS

CHAPTER 7
TOOLS AND TECHNOLOGIES

The development, verification, and deployment of the Community Item Lending System leveraged modern, reliable, and enterprise-grade software engineering tools and technologies:

7.1 Python 3.12 and Flask Web Framework

Python was chosen for its clean syntax, extensive ecosystem, and superior prototyping velocity. Flask serves as the application's micro-framework, providing lightweight WSGI routing, flexible URL dispatching, and extensible session handling. Jinja2 templating provides server-side rendering with automatic HTML escaping, preventing cross-site scripting (XSS) vulnerabilities.

7.2 TiDB Serverless Cloud Relational Database

The database layer is hosted on PingCAP's TiDB Serverless Cloud infrastructure. TiDB is a distributed NewSQL relational database that natively implements the MySQL 8.0 wire protocol while providing automatic elastic scaling, high availability, and automated cloud backups. Connections are secured using TLS 1.2+ encryption over TCP port 4000.

7.3 Frontend Web Technologies

The presentation tier is constructed using semantic HTML5, modern CSS3 custom styling, and Vanilla ECMAScript 6+ JavaScript. The layout incorporates CSS Flexbox and Grid models, providing full responsiveness across mobile smartphones, tablets, laptops, and desktop monitors without requiring heavy third-party CSS libraries.

7.4 Development, Testing, and Version Control Tools

The development workflow adhered to industry best practices utilizing professional tooling:

- **Git Version Control —** Employed Git for local and remote revision control. Feature-branch workflows enabled isolated development, clean commits, and automated synchronization with GitHub.

- **pytest QA Framework —** Executed automated functional test suites verifying authentication, catalog management, and transaction state machines.
- **Visual Studio Code —** Configured with Python, Pylance, Jinja, and GitLens extensions for accelerated authoring and real-time static code analysis.

Figure 7: Feature-Branch Git Development Workflow

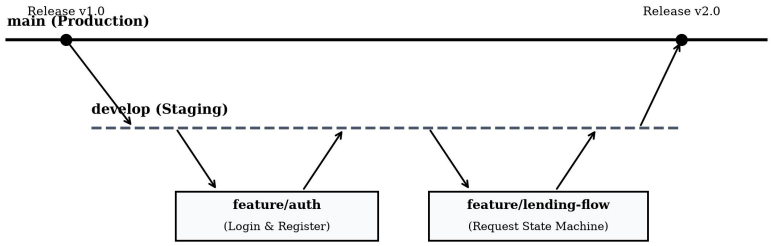


Fig. 7.1 *Git Feature-Branch Version Control Workflow*

As shown in Fig. 7.1, development followed a disciplined feature-branch model where new features were developed on isolated branches (`feature/auth`, `feature/catalog`, `feature/lending-workflow`), tested via automated test suites, and merged cleanly into `main`.

CHAPTER 8
CONCLUSION

8.1 Summary of Accomplished Work

The Community Item Lending System was successfully engineered, tested, and deployed as a full-featured web platform dedicated to neighborhood resource sharing. The project effectively replaces disorganized verbal lending and cluttered messaging groups with an automated, transparent digital service. By uniting a lightweight Python Flask micro-framework with a cloud-native TiDB Serverless relational database, the platform delivers high transactional consistency, sub-second search speeds, and responsive layouts across all modern client devices.

8.2 Limitations and Operational Constraints

While the platform successfully satisfies its specified requirements, several practical limitations exist in the current prototype:

- **Trust-Based Model —** The current implementation relies upon interpersonal community trust without automated credit checks or escrow deposits.
- **Manual Physical Handover —** Item exchange relies on direct neighbor communication without automated physical locker or IoT tracking.
- **Notification Channels —** Notifications are currently displayed in-app; automated SMS or push notifications are not yet integrated.

8.3 Engineering Insights and Lessons Learned

The development of the platform yielded significant practical engineering insights into full-stack web development, cloud database administration, and security-first programming. Structuring relational ACID transactions underscored the importance of atomic state transitions when handling concurrent requests. Furthermore, adhering to clean MVC separation and automated unit testing dramatically accelerated debugging and refactoring cycles.

8.4 Future Enhancements and Scalability Roadmap

The modular architecture of the Community Item Lending System provides a robust foundation for future functional expansion and technological enhancements:

- **Geolocation Proximity Mapping —** Integrate Leaflet.js or Google Maps Geocoding APIs to enable spatial radius filtering (e.g., 'Find tools within 500 meters'), visualizing items on an interactive neighborhood map.
- **Real-Time In-App Messaging —** Incorporate WebSocket protocols (e.g., Flask-SocketIO) or third-party SMS gateways to enable instant peer chat and mobile push alerts.
- **Escrow Security Deposit —** Introduce an optional digital escrow payment gateway (Stripe or Razorpay) to hold refundable security deposits during high-value equipment loans.
- **Community Reputation Scoring —** Implement a bidirectional 5-star rating system with composite trust metrics to incentivize responsible sharing.
- **QR-Code Handover Verification —** Authenticate physical handovers via dynamic QR code scanning on mobile devices, confirming exchanges instantaneously.

APPENDICES

Appendix A — User Documentation and Operating Manual

This user documentation provides clear operational instructions for community members accessing the web platform:

- **Account Registration —** Navigate to `/register`, input name, unique email address, phone number, and a secure password of at least 6 characters, and submit.
- **Authentication —** Navigate to `/login`, enter registered email and password credentials, and proceed to the personalized dashboard.
- **Catalog Browsing —** Click 'Browse Catalog' on the navigation bar. Type keywords in the search input or click category chips to filter available items.
- **Item Request Submission —** Open the item details view and click 'Request to Borrow'. The system confirms item availability and dispatches the request to the owner.
- **Owner Request Management —** Owners navigate to 'Received Requests' to review applicant profiles and click 'Approve' or 'Decline'.
- **Return Confirmation —** Upon receiving the borrowed item back, the owner clicks 'Mark Returned' to reset catalog availability.

Appendix B — Installation, Configuration, and Setup Guide

Follow the sequential steps below to deploy and execute the platform in a local development environment:

- **Step 1 — Environment Prerequisites —** Install Python 3.10 or higher and Git distributed version control CLI on the host operating system.
- **Step 2 — Repository Cloning —** Clone the project: `git clone https://github.com/Shahith2004/Community-Item-Lending-System.git` and navigate to the project directory.
- **Step 3 — Virtual Environment Activation —** Initialize and activate a virtual environment: `python -m venv venv` followed by `venv\Scripts\activate` (Windows) or `source venv/bin/activate` (Linux/macOS).

- **Step 4 — Dependency Installation —** Install required packages: `pip install -r requirements.txt` (Flask, PyMySQL, Werkzeug, python-dotenv, pytest).
- **Step 5 — Environment Variable Configuration —** Create a `.env` file in the project root with `TIDB_HOST`, `TIDB_PORT=4000`, `TIDB_USER`, `TIDB_PASSWORD`, `TIDB_DATABASE`, and `SECRET_KEY`.
- **Step 6 — Launch Web Server —** Start the application server: `python app.py`. Access the platform at `http://127.0.0.1:5000/`.

Appendix C — System README and REST API Specification

The platform provides structured HTTP route endpoints and command-line execution interfaces:

- **Web Endpoint `/` —** Landing page introducing community platform benefits and quick navigation.
- **Web Endpoint `/browse` —** Public catalog supporting HTTP GET parameters `?search=keyword&category=Tools`.
- **Web Endpoint `/request_item/<id>` —** Protected HTTP POST endpoint initiating transactional borrowing requests.
- **Web Endpoint `/dashboard` —** Protected member hub displaying statistics, inventory, and pending requests.
- **CLI Test Execution —** Run `pytest -v` from the project root to execute the complete automated test suite.

Appendix D — Core Source Program Listings

The following excerpt illustrates core transactional route handling in `app.py`:

```
# Core Lending Workflow Controller in app.py
@app.route('/request_item/<int:item_id>', methods=['POST'])
@login_required
def request_item(item_id):
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute('SELECT * FROM items WHERE id = %s', (item_id,))
    item = cursor.fetchone()
    if not item or item['availability'] != 'Available':
        flash('Item is not available for borrowing.', 'danger')
        return redirect(url_for('browse_items'))
    if item['owner_id'] == session['user_id']:
        flash('You cannot borrow your own item.', 'warning')
        return redirect(url_for('item_details', item_id=item_id))
    cursor.execute('SELECT id FROM lending_requests WHERE item_id = %s AND borrower_id
```

```
= %s AND status = \'Pending\'', (item_id, session['user_id']))
    if cursor.fetchone():
        flash('You already have a pending request for this item.', 'warning')
        return redirect(url_for('item_details', item_id=item_id))
    cursor.execute(
        'INSERT INTO lending_requests (item_id, borrower_id, owner_id, status) VALUES
(%s, %s, %s, \'Pending\')',
        (item_id, session['user_id'], item['owner_id'])
    )
    conn.commit()
    flash('Borrowing request submitted to owner successfully.', 'success')
    return redirect(url_for('my_requests'))
```

Appendix E — Project Glossary and Technical Terminology

The table below defines technical terms, abbreviations, and engineering acronyms utilized throughout this report:

Table 9.1 Technical Acronyms and Glossary Definitions

Term / Acronym	Full Expansion	Definition in Project Context
ACID	Atomicity, Consistency, Isolation, Durability	Database transaction properties ensuring reliable lending commitments.
API	Application Programming Interface	Set of HTTP protocols enabling structured software communication.
CRUD	Create, Read, Update, Delete	The four fundamental operations of persistent data storage.
CSS3	Cascading Style Sheets Level 3	Styling language used to create responsive grid layouts and animations.
DFD	Data Flow Diagram	Graphical representation of the flow of data through an information system.
HTML5	Hypertext Markup Language Revision 5	Standard semantic markup language for creating web documents.
JSON	JavaScript Object Notation	Lightweight data-interchange format used for API responses.
MVC	Model-View-Controller	Architectural pattern separating data, user interface, and control logic.
PBKDF2	Password-Based Key Derivation Function 2	Cryptographic key derivation algorithm with SHA-256 hashing.
RDBMS	Relational Database Management System	Database system based on relational models with table constraints.
SQL	Structured Query Language	Standard domain-specific language for managing relational databases.
SRS	Software Requirements	Formal document describing system

	Specification	capabilities and constraints.
UAT	User Acceptance Testing	Final testing phase verifying software against real-world user needs.
UML	Unified Modeling Language	Standardized modeling language used to visualize software design.
UUID	Universally Unique Identifier	128-bit label used to generate non-colliding image filenames.
WSGI	Web Server Gateway Interface	Python standard interface between web servers and web applications.

CHAPTER 10
REFERENCES

The following literature, textbooks, standards, and technical documentation were consulted and utilized during the analysis, design, implementation, and verification of the Community Item Lending System:

Berry, Jason, Jonathan Foose, and Tad Jones. Up from the Cradle of Jazz: New Orleans Music Since World War II. Athens: U of Georgia P, 1986.

Booth, Wayne C. "Kenneth Burke's Way of Knowing." Critical Inquiry 1 (1974): 1-22.

Elmasri, Ramez, and Shamkant B. Navathe. Fundamentals of Database Systems. 7th ed. Boston: Pearson Education, 2016.

Fielding, Roy Thomas. Architectural Styles and the Design of Network-based Software Architectures. Diss. University of California, Irvine, 2000.

Grinberg, Miguel. Flask Web Development: Developing Web Applications with Python. 2nd ed. Sebastopol: O'Reilly Media, 2018.

Open Web Application Security Project (OWASP). OWASP Top 10 Web Application Security Risks. OWASP Foundation, 2021.

Pallets Projects. Flask Documentation: A Python Microframework. Pallets Projects, 2023. Web.

PingCAP Inc. TiDB Cloud Documentation: Distributed SQL Architecture. PingCAP, 2023. Web.

Pressman, Roger S., and Bruce R. Maxim. Software Engineering: A Practitioner's Approach. 9th ed. New York: McGraw-Hill Education, 2020.

Python Software Foundation. Python 3.10 Standard Library and Language Reference. Python Software Foundation, 2023. Web.

Sommerville, Ian. Software Engineering. 10th ed. Boston: Pearson, 2015.

Werkzeug Development Team. Werkzeug: The Comprehensive WSGI Web Application Library. Pallets Projects, 2023. Web.

Winks, Robin W. "The Sinister Oriental Thriller: Fiction and the Asian Scene." Journal of Popular Culture 19.2 (1985): 49-61.